Technical Report For Granite Guardian

Ines Mouine - smjg43
Krisha Chauhan - mknf66
Rayane El-Mselmi - xvns66
Adam Fallon - vtwn33
Ewan Wong - fvcg54
Adel - :)

Section 1: Introduction

1.1 Project summary: what is Granite Guardian and what does it do?

For an average driver, the dashboard of a car can be a source of stress. When a "Check Engine" light flickers on, it rarely provides context, leaving the user caught between the fear of a breakdown and the confusion of technical jargon. **Granite Guardian** was born from the desire to bridge this gap, turning cold, complex vehicle data into a supportive and transparent conversation.

Our system allows users to upload **OBD-II** CSV files. Once uploaded, the platform performs an analysis of the vehicle's vital signs (ex: engine RPM, catalytic converter temperatures, or coolant temperature). The client requested CSV files, though this system could easily be extended to work with live data.

Instead of just flagging a numerical anomaly, our integration of the **IBM Granite 3.0 2B** model allows the system to speak to the user, explaining why a value is concerning and what steps should be taken next. The goal is to move from reactive repairs to proactive care and saving users money and providing peace of mind

1.2: Access and Technical Setup

The system was developed to run entirely on the user's local machine or on an embedded system. This ensures that sensitive vehicle diagnostic logs (which could technically reveal driving patterns or locations) never leave the user's hardware, and it mitigates the need for a constant Wi-Fi connection.

- The frontend was developed using React 19 and **TypeScript** which provides a "chic," high-contrast dashboard designed for readability in various lighting conditions. It utilizes Lucide-React for intuitive iconography.
- The backend is powered by a FastAPI server that handles data validation and diagnostic logic. It uses SQLAlchemy as an ORM to communicate with a local SQLite database.
- Rather than relying on cloud based LLMs that require low-latency internet connections and monthly subscriptions, we used Ollama to serve the Granite3.02B model locally. This specific model was chosen because it was the best version that balances automotive diagnostic accuracy and the ability to run quickly and smoothly on consumer hardware. Theoretically, it would be possible to swap out this model with a larger or smaller one depending on the capabilities of a user's hardware.

<u>About the required Environment for Launch:</u>

To ensure the system operates correctly, you must activate the following environment:

- **Python 3.12+** Virtual Environment containing dependencies like fastapi, uvicorn and pandas.
- The **Ollama** application must be running in the system tray to handle AI requests.
- And **Node.js** Environment: To serve the React frontend via Vite

1.3: Status of Behavioural Requirements

ID	Requirement	State	Justification /Commentary
BR1	CSV File Ingestion	**Completed**	allows for uploading CSV files.
BR2	Anomaly detection	**Completed**	detects anomalies in engine coolant sensor (e.g. noise).
BR3	Granite AI Chatbot	**Completed**	local integration via Ollama for natural language explanations.
BR4	Dashboard	**Not Complete**	graphical representation of critical sensor data for user diagnostics

Table 1: Status of Behavioral Requirements

Table 1 provides an overview of the system's core functionalities as defined in the Requirements Specification. It tracks the successful implementation of each feature, ensuring alignment between the initial user stories and the final software deliverable

Section 2: Technical Development

2.1 Technologies, Platforms and System Architecture

To fulfill the requirements (delivering a responsive and secure application), Granite Guardian was built using a modern three-tier architecture. This separation ensures that the user interface, business logic and data storage can operate and scale independently:

1. The frontend

The user interface is developed as a single page application in which we included:

- React for its efficient, component-based architecture.
- Typescript to enforce type checking and thus reduce runtime errors during development.
- The usage of React for its component-based architecture which allowed us to build reusable UI elements (such as the Chatbot interface and the data dashboards) and Typescript to enforce type-checking to reduce runtime errors during development.

2. The backend

- FastAPI.
- Ollama for hosting Granite model.
- SQLAlchemy as an ORM to interact with a SQLite database.
- Numpy/pandas for handling input CSV data.
- Scikit-learn for training ML classifiers.

3. The Diagnostic & AI Inference Layer

The AI engine is hosted entirely on the user's hardware (rather than relying on external APIs):

- Ollama Framework: used to serve as the local execution environment for our LLM (large language model).
- IBM Granite 3.0 2B (granite3-dense:2b): We chose ot base the Chatbot on this model so when the backend detects a mechanical anomaly, it constructs a prompt containing the error context and sends it to the local Granite model. The model then generates a natural-language explanation tailored for a non-expert driver, which is routed back to the React frontend.

4. The Data Persistence Layer

To store the users sessions, his historic of its diagnostic alerts and system configurations we implemented the following solutions:

- **SQLite**: Chosen because our system emphasizes local deployment ("Privacy by Design"), SQLite allows the entire database to be stored as a single local file (.db) without requiring the user to configure complex external database servers (ex PostgreSQL, MySQL..).
- And the SQLAlchemy: we used it as our Object/Relational Mapper. It abstracts the raw SQL queries into Python objects, preventing SQL injection attacks and ensuring the data access layer remains secure and maintainable.

Section 3: Use Instructions

3.1 Installation and System Requirements

To ensure the system operates efficiently, particularly the local Large Language Model (LLM), the host machine must meet the following hardware and software requirements:

- **Operating System**: macOS (Apple Silicon M1/M2/M3 or Intel), Windows 10/11 or Linux
- **Minimum Hardware**: 8 GB RAM, multi-core CPU (Intel i5/Ryzen 5 or equivalent) and at least 5 GB of available storage (we recommend 16 GB RAM and an SSD for optimal local AI inference speeds)

- **Prerequisite Software:** Python 3.12+, Node.js (v18+), npm and Ollama

3.2 Deployment and Configuration

The deployment process involves setting up three distinct components: the backend API, the local database and the frontend client and were going to give you the steps:

Step 1: Backend Setup & Database Initialization (correct me please)

Navigate to the Backend directory in your terminal: cd Backend

Create and activate a Python virtual environment to isolate dependencies:

- Mac/Linux: “python3 -m venv.venv” then “source.venv/bin/activate”
- Windows: “python -m venv.venv” then “.venv\Scripts\activate”

Install the required packages: “pip install fastapi uvicorn pandas numpy pydantic httpx sqlalchemy ollama”

Step 2: AI Model Deployment

To deploy the local AI securely, launch the Ollama application on your system. Once the application is running in the background, open a terminal and execute the following command to download the specific Granite model: “ollama run granite3-dense:2b”

Step 3: Frontend Setup

Go to the root directory and install the Node modules: “npm install”

3.3 Launching the Application

Once you’re done with the previous part, you can launch the application and perform the initial setup which consist in:

1. Start the backend server by going into the backend directory through the “cd backend” command. and run : “python -m uvicorn main:app --reload”
2. Open a new terminal within the frontend directory to start the frontend server thrw: “npm run dev”
3. access the system by opening a web browser and navigate to: “http://localhost:5173”
4. Upon launching, the user will get an authentication screen. Click "Register" to create a new user account. The system will securely hash your password and generate a JWT for your session. Once logged in, you will be redirected to the main diagnostic dashboard where you can upload your first OBD-II file (reminder: It has to be a csv file)